Build Plan

Brain Layer Prompt Package Plan

Goal

Build a production-ready prompt and artifact set you can paste into gemini so it generates a deterministic, implementation-grade brain layer between frontend and Supabase backend, with explicit execution rules and no vague language.

Deliverables I Will Draft

- A `Master System Prompt` for gemini that enforces strict output in the 7 required sections and blocks non-compliant responses.
- An `Input Packet Template` you fill with your knowledge bases, frameworks, features, and workflows.
- A `Determinism Contract` that forces formulas, thresholds, JSON schemas, pseudocode, and fixed ordering.
- A `Non-Technical Language Layer` that rewrites labels/instructions into operator-friendly wording while preserving machine-readable structures.
- A `Supabase Integration Contract` covering tables, vectors, retrieval metadata, workflow states, memory logs, and versioning fields.
- A `Frontend Handoff Contract` for `cto.new` + Cloudflare that defines request/response payloads, status lifecycle, and rendering-ready outputs.
- A `gemini Output Validator Checklist` to verify each section is executable and complete before accepting output.

Implementation Sequence

1. Define strict output contract and response guardrails.
    - Enforce exact 7-section headings.
    - Enforce required artifacts per section (schema, formulas, thresholds, templates).
    - Enforce banned patterns (`could`, optional language, vague descriptors).
2. Define canonical input model.
    - Normalize all user-provided material into one canonical JSON envelope.
    - Add required fields and null-handling rules.
    - Add deterministic defaults for missing values.
3. Define section-by-section generation rules.
    - Section 1: classification labels, weighted scoring formulas, deterministic thresholds.

    - Section 2: chunking, embedding metadata, retrieval pipeline, ranking formula.
    - Section 3: priority/risk/confidence scoring math and fixed execution order.
    - Section 4: framework-to-executable transform template.
    - Section 5: workflow state machine JSON with trigger/retry/fallback formulas.
    - Section 6: memory schema + learning/update equations and update cadence.
    - Section 7: single end-to-end trace with all intermediate states.
4. Define Supabase-ready data and query contract.
    - Table-level schema spec for documents, chunks, embeddings, decisions, workflow runs, memory events.
    - Deterministic key strategy and timestamps.
    - Retrieval and filtering fields required for RAG and decision traceability.
5. Define frontend integration contract.
    - API payload schemas for submit, status poll, and result retrieval.
    - Machine-readable + human-readable output split for non-technical users.
    - Error surface contract and user-safe fallback messages.
6. Create final paste-ready gemini package.
    - `SYSTEM` block.
    - `INPUT TEMPLATE` block.
    - `OUTPUT FORMAT` block.
    - `VALIDATION CHECKLIST` block.
    - `WAIT FOR INPUT` terminal instruction.

Acceptance Criteria

- gemini output is strictly in the 7 required sections, in order.
- Every section contains executable structures (JSON schemas, formulas, pseudocode, thresholds).
- Supabase storage and retrieval fields are concrete and queryable.
- Workflow logic includes explicit states, transitions, retries, and fallbacks.
- Language is understandable by non-technical operators without removing execution precision.
- No summary/fluff language appears in generated artifacts.

Notes on Scope

- This plan produces the complete prompt/instruction package and templates.
- After you provide your raw product inputs, the same package can be used to generate the full brain-layer spec instance for your product.

Tab 5

What You Build Next (In Order)

You already have:

- Backend (Supabase)
- Some edge functions / triggers
- Tools (Cursor, cto.new)

Step 1 — Build the Input Classifier (Your Entry Brain)

This is the **first real "thinking" component**

What it does:

Every user input gets classified into a type:

```
{
 "type": "ticket | action | question | automation",
 "priority": "low | medium | high",
 "domain": "tech | pmo | ops",
 "intent": "diagnose | create | fix | analyze"
}
```

Where it lives:

- Supabase Edge Function OR Cloudflare Worker

What it calls:

- gemini (best for structured outputs)

Why this matters

Without this:

- everything is random
- no routing
- no workflows

With this:
👉 your system becomes controllable

Step 2 — Build the Context Builder (Memory Injection Layer)

Now that you know *what the input is*, you fetch the **right data**

It pulls from Supabase:

- knowledge base
- past tickets
- system logs
- workflows
- user context

Output:

```
{
 "input": "...",
 "classification": {...},
 "context": {
   "relevant_knowledge": [...],
   "similar_cases": [...],
   "system_state": {...}
 }
}
```

Why this matters

LLMs are dumb without context.

👉 This is what makes your system feel “intelligent”

Step 3 — Build the Prompt Engine (Core Thinking Layer)

This is the MOST important piece.

It dynamically builds prompts like:

You are MARTIN OS.

INPUT:
{{user_input}}

TYPE:
{{classification}}

CONTEXT:
{{retrieved_data}}

TASK:
- Diagnose
- Recommend fix
- If possible, generate executable steps

OUTPUT FORMAT:
JSON ONLY

This is where you use:

- ChatGPT (design + iteration)
- gemini (execution)

Step 4 — Build the Decision Engine (Router)

Now you take the LLM output and decide:

What happens next?

IF type = "ticket"
→ send to diagnostic workflow

IF type = "action"
→ generate execution steps

IF type = "automation"
→ create new workflow

IF confidence < threshold
→ escalate to human

This is your REAL “brain”

Without this:

- your app is just a chatbot

With this:
👉 it becomes an **operator system**

Step 5 — Build the Execution Layer (Do the Work)

Now your system actually **does things**

Examples:

- create/update rows in Supabase
- trigger edge functions
- run fixes
- generate tasks
- update statuses

Example:

```
{
 "action": "create_ticket",
```

"data": {
  "title": "API failure",
  "steps_to_fix": [...]
}
}

Step 6 — Build the Memory Loop (Learning System)

Every result gets saved:

- input
- classification
- decision
- output
- success/failure

👉 stored in Supabase

Why this matters

This is how your system becomes:

- smarter over time
- more accurate
- less dependent on prompts

The Full Flow (Your System)

User Input
↓
Classifier (gemini)
↓
Context Builder (Supabase queries)
↓
Prompt Engine

↓
LLM (gemini / ChatGPT)
↓
Decision Engine
↓
Execution Layer
↓
Memory Storage (Supabase)

What You Should Do TODAY

Don’t try to build everything at once.

Do this in order:

✅ 1. Build the classifier edge function

✅ 2. Make it return structured JSON

✅ 3. Store results in Supabase

Then:

✅ 4. Add context retrieval

✅ 5. Build ONE workflow (ticket handling)

The Mistake to Avoid

Do NOT:

- keep adding tools
- overbuild UI
- rely on one giant prompt

Reality Check

Right now, you don't have a system yet.

You have:
👉 infrastructure + potential

What you're building next is:
👉 **behavior**

If You Want — I Can Do This For You

I can give you:

- exact **Edge Function code (copy/paste)**
- exact **gemini prompts for classifier + reasoning**
- exact **Supabase table structure for memory**
- a **working "ticket → diagnosis → fix" pipeline**

Just say:

> "Build my classifier + first workflow"

And I'll wire the whole thing up cleanly.

where do I put these prompts (also, im not using gemini - its not free, lets use gemini and chatgpt instead)

Good call on using **free models** — you can absolutely do this with

- Gemini
- ChatGPT

Now let's answer your actual question:

Where Do the Prompts Go?

👉 **They DO NOT go in one place**

You will store and use prompts in **3 different layers** depending on their role.

---

1. Execution Prompts → Inside Edge Functions (PRIMARY)

This is the most important place.

Location:

- Supabase Edge Functions
  OR
- Cloudflare Workers

What goes here:

- classifier prompt
- reasoning prompt
- workflow prompt

Example (REAL structure):

```
const prompt = `
You are MARTIN OS.

Classify the input into:
- type: ticket | action | question | automation
- priority: low | medium | high
- intent: diagnose | create | fix | analyze

Return ONLY JSON.

Input:
${user_input}
`;
```

Then:

callGemini(prompt)

Why here?

Because this is where:

- logic runs
- decisions happen
- workflows start

👉 This = your **actual thinking layer**

---

2. Prompt Templates → Stored in Supabase (DYNAMIC BRAIN)

Location:

- Supabase tables

Create a table like:

`prompts`

id	name	content	version
1	classifier	"You are..."	v1
2	diagnostic	"Analyze..."	v1

Then your edge function does:

```
const promptTemplate = getPromptFromDB("classifier");

const prompt = promptTemplate.replace("{{input}}", user_input);
```

Why this matters

Without this:

- every change = redeploy code

With this:
👉 you can update prompts **live without touching code**

---

3. Design / Testing Prompts → ChatGPT (YOUR WORKSHOP)

Use ChatGPT to:

- design prompts
- refine outputs
- test structures

Workflow:

1. Test prompt in ChatGPT
2. Improve structure
3. Copy into:
    - Supabase (template)
    - Edge Function (execution)

Important: ChatGPT is NOT your system

It's just your:
👉 prompt lab

How It All Connects

User Input
↓
Edge Function
↓
Pull Prompt Template (Supabase)
↓
Inject Context + Input
↓
Send to Gemini / ChatGPT API
↓
Receive Response
↓
Store Result (Supabase)

What You Should Do Next (Concrete)

Step 1 — Create a `prompts` table in Supabase

Start with:

- `classifier`
- `diagnostic`
- `action_generator`

Step 2 — Build ONE Edge Function

It should:

1. take input
2. load prompt from DB
3. call Gemini
4. return JSON

Step 3 — Hardcode FIRST, Then Upgrade

At first:

```
const prompt = "hardcoded...";
```

Once it works:

👉 move it to database

Simple Rule (Don’t Overthink This)

👉 Prompts that RUN = Edge Functions
👉 Prompts that CHANGE = Database
👉 Prompts you TEST = ChatGPT

Common Mistake You’re About to Avoid

Most people:

- dump everything into one giant prompt
- keep it inside the UI
- don’t structure outputs

You’re doing the opposite:

👉 modular prompts
👉 controlled execution
👉 real system design

tab3

You are an elite principal systems architect and full-stack AI engineer. Build the “Brain Layer” for a multi-product operations platform running on ONE Supabase database, with shared core tenancy + 3 domain schemas: pmo_ops, tech_ops, miidle.

==========
PRODUCT CONTEXT
==========
This is an AI-native operations command ecosystem with:
1) PMO Ops Command Center (strategy + execution + diagnostics + advisory)
2) Tech Ops (reliability + incidents + SLO + runbooks + automation)
3) Miidle (workflow + knowledge + automation + product operations)

The platform is multi-role (owner/admin/member) and multi-tenant (organization-scoped).
The central objective is to convert user inputs into intelligent, auditable, framework-driven actions.

==========
NON-NEGOTIABLE ARCHITECTURE
==========
Use one Supabase project / one Postgres database with:
- Shared core in public schema:
  - profiles (1:1 auth.users)
  - organizations
  - organization_members
  - helper functions: is_org_member(org_id, user_id), is_org_admin(org_id, user_id)
- Domain schemas:
  - pmo_ops
  - tech_ops
  - miidle
- RLS enabled on all user-facing tables.
- Org boundary enforced everywhere via RLS helpers.
- UUID PKs, created_at, updated_at conventions.
- JSONB metadata columns for extensibility.
- pgvector enabled for retrieval.

==========
BRAIN LAYER (MIDDLE LAYER) - REQUIRED ENGINES
==========
Implement these 5 systems in each domain (with domain-specific labels/data), keeping table/function naming as consistent as possible:

1) INPUT CLASSIFICATION ENGINE
Function:
classify(input) -> {

```
  type: "ticket" | "action" | "question" | "idea" (domain-specific variants allowed),
  domain: "PMO" | "Tech" | "General",
  priority: 1-5,
  complexity: "low" | "medium" | "high"
}
```

Scoring formula:
Score = (IntentWeight * IntentMatch) + (KeywordWeight * KeywordMatch) + (ContextWeight * ContextMatch)

2) KNOWLEDGE RETRIEVAL ENGINE (RAG CONTEXT BUILDER)
Function:

```
retrieve_context(input, classification) -> {
  documents: [],
  frameworks: [],
  past_cases: []
}
```

Similarity formula:
Similarity(A,B) = (A dot B) / (||A|| * ||B||)

Requirements:
- Store embeddings in pgvector
- top-k retrieval
- reranking-ready design
- support knowledge docs, framework docs, and past cases

3) DECISION ENGINE (CORE BRAIN LOGIC)
Function:

```
decide(input, context, classification) -> {
  action_plan: [],
  tools_to_use: [],
  confidence_score: 0-1
}
```

Must include:
A) Rule-based deterministic logic
B) Framework execution (IP layer)
C) Scoring formulas

Required formulas:
Priority = Impact * Urgency * Frequency
Confidence = (RelevantData + PastSuccess + ModelCertainty) / 3
Risk = Probability * Impact

4) ACTION / WORKFLOW ORCHESTRATOR
Function:

```
execute(action_plan) -> {
  steps: [],
  status: "success" | "fail" | "pending"
}
```

Must include:
- workflow definition model with trigger + steps
- state machine:
  State_next = f(State_current, Action)
- retry/fallback logic
- escalation handling

5) MEMORY + LEARNING SYSTEM
Function:
store_result(input, output, success)

Learning update formula:
NewWeight = OldWeight + alpha * (Outcome - Prediction)

Persist at minimum:
- inputs
- decisions
- outputs
- success/failure
- time-to-resolve
- reusable patterns

==========
END-TO-END FLOW
==========
INPUT
-> CLASSIFY
-> RETRIEVE_CONTEXT
-> DECIDE
-> EXECUTE
-> STORE_AND_LEARN
-> OUTPUT_TO_UI

==========
SHARED TABLE NAMING PATTERN (APPLY PER SCHEMA)
==========

classification_rules
classification_runs
classification_labels
retrieval_runs
decision_rules
decision_runs
decision_outcomes
workflows
workflow_steps
workflow_runs
workflow_state_transitions
retry_policies
interactions
decisions_memory
outcomes
patterns
learning_updates
knowledge_documents
knowledge_chunks
past_cases
formula_registry
edge_function_registry (or equivalent automation registry)

==========
PMO_OPS DOMAIN REQUIREMENTS
==========
Core PMO entities:
departments, initiatives, initiative_dependencies, risks, capacity_snapshots, process_metrics
signals, diagnoses, advisories, structural_remedies, insight_cards
frameworks, framework_rules, framework_evaluations
maturity_scores, priority_scores, score_inputs, score_runs

Seed framework baseline:
Porter, Rumelt, Balanced Scorecard, OKRs, Lean, Six Sigma, TOC
(and advanced methods references like MCDA/AHP/ANP, Bayesian, OR/LP/IP, systems thinking)

PMO signal categories:
capacity_overload
execution_delay
initiative_conflict
dependency_bottleneck
strategic_misalignment
risk_escalation

performance_anomaly

==========
TECH_OPS DOMAIN REQUIREMENTS
==========
Core entities:
services, environments, service_dependencies, incidents, incident_timeline
alerts, slo_policies, error_budgets, deployments, change_events
runbooks, automation_jobs, automation_runs, postmortems

Tech workflow states example:
NEW -> TRIAGED -> MITIGATING -> RESOLVED -> POSTMORTEM

Track reliability metrics:
availability, latency, MTTR, MTBF, incident severity trends, deployment health

==========
MIIDLE DOMAIN REQUIREMENTS
==========
Core entities:
workspaces, programs, assets, work_items, workflow_states
playbooks, knowledge_bases, knowledge_documents, knowledge_chunks
formula_registry, function_catalog, event_trigger_catalog, edge_function_registry

Workflow states example:
NEW -> IN_PROGRESS -> REVIEW -> RESOLVED -> ESCALATED

==========
SECURITY / GOVERNANCE REQUIREMENTS
==========
- RLS on all domain tables.
- Member read and scoped write where appropriate.
- Admin/owner elevated write controls.
- No cross-org data leakage.
- Service-role-only pathways for system-generated pipeline records where necessary.
- Auditable run logs for each engine (classification/retrieval/decision/execution/learning).

==========
WHAT TO GENERATE
==========
Generate production-ready artifacts in this exact order:
1) SQL Block A: shared core (public schema, helper functions, profile trigger, RLS)
2) SQL Block B: pmo_ops schema + brain tables + PMO core + indexes + RLS + seeds
3) SQL Block C: tech_ops schema + brain tables + reliability core + indexes + RLS + seeds

4) SQL Block D: miidle schema + brain tables + workflow/knowledge core + indexes + RLS + seeds
5) SQL Block E: cross-domain seed data + validation queries
6) Edge function stubs for:
  - classify
  - retrieve_context
  - decide
  - execute
  - store_result
7) API contract examples (JSON payloads) for each stage
8) RLS test matrix (owner/admin/member and cross-org denial cases)
9) Migration order + rollback strategy
10) “Strict mode” hardening recommendations:
  - policy idempotency
  - audit tables
  - soft-delete strategy
  - state transition constraints
  - performance indexes and explain-plan checks

==========
IMPLEMENTATION STYLE
==========
- Be explicit, deterministic, and production-focused.
- Avoid pseudo-architecture without concrete schema/SQL.
- Prefer idempotent migration-safe SQL patterns where possible.
- Use concise comments in SQL explaining non-obvious logic.
- Ensure generated output is copy/paste-ready.

Now produce the full implementation.

Tab 2

PMO Ops Production-Ready Template Plan

Scope

Build a complete Supabase-ready template that operationalizes the PMO Ops plan into an implementation bundle:

- SQL foundation (schema, constraints, RLS, indexes, helper functions)
- Domain seed data (frameworks, formulas, function catalog, trigger/event catalog, knowledge bases)
- Scoring + reasoning pipeline shape (Signal, Diagnosis, Advisory, Structural Remedies)
- Migration/run order and production checklist

Primary references:

- [PMO Ops Command Center Full Plan Based On TAR.pdf](/Users/apphia/Library/Application Support/Cursor/User/workspaceStorage/de11a0caf178e8b1fda6f38373076257/pdfs/dd9fd626-e374-4b44-aeb5-500731c42239/PMO Ops Command Center Full Plan Based On TAR.pdf)
- [CURRENT_ PMO-Ops App Plan - v3.pdf](/Users/apphia/Library/Application Support/Cursor/User/workspaceStorage/de11a0caf178e8b1fda6f38373076257/pdfs/87729f48-76ea-4662-8359-b848b7780ad5/CURRENT_ PMO-Ops App Plan - v3.pdf)

Target Data Model (Production Template)

- Core identity/tenant:
  - `profiles`, `organizations`, `organization_members`, role matrix
- Portfolio operations:
  - `departments`, `initiatives`, `initiative_dependencies`, `risks`, `capacity_snapshots`, `process_metrics`
- Reasoning pipeline:
  - `signals`, `diagnoses`, `advisories`, `structural_remedies`, `insight_cards`
- Scoring system:
  - `maturity_dimensions`, `maturity_scores`, `priority_scores`, `score_inputs`, `score_runs`
- Framework intelligence:

    - `frameworks`, `framework_rules`, `framework_mappings`, `framework_evaluations`
- Knowledge + playbooks:
    - `knowledge_bases`, `knowledge_documents`, `kb_topics`, `kb_chunks` (embedding-ready metadata)
- Automation catalog:
    - `app_functions_catalog`, `formula_registry`, `event_trigger_catalog`, `edge_function_registry`

Security and Multi-Tenant Controls

- Enable RLS across all user-facing tables.
- Use org membership helpers (`is_org_member`, `is_org_admin`) for policy checks.
- Restrict cross-tenant reads/writes, with explicit org-scoped joins.
- Keep service-role-only write paths for pipeline run tables where needed (`score_runs`, system-generated insight rows).

Seed Strategy (Business-Aligned)

- Seed canonical frameworks from docs:
    - Porter, Rumelt, Balanced Scorecard, OKRs, Lean, Six Sigma, TOC
    - Additional advanced methods referenced in plan (MCDA/AHP/ANP, Bayesian, OR/LP/IP, systems thinking)
- Seed maturity dimensions and default weights:
    - Strategic Alignment, Execution Discipline, Operational Capacity, Process Structure, Risk Management
- Seed function + formula templates for:
    - Maturity score (0-100), executive priority score, dependency bottleneck scoring, risk escalation signals
- Seed event/trigger catalog entries aligned to plan:
    - capacity overload, execution delay, dependency bottleneck, strategic misalignment, risk escalation

Execution Bundle Deliverables

- One copy-paste SQL block for initial production template:
    - extensions, tables, constraints, indexes, triggers, helper functions, RLS policies, core seed rows
- One copy-paste SQL seed block for realistic sample org data:
    - departments, initiatives, dependencies, risks, score inputs, sample insights
- One operational checklist section:

    - migration order, validation queries, RLS test matrix, environment requirements for Edge Functions

Build Flow (Aligned to Phases in PDF)

```
flowchart TD
  phase0[Phase0Foundations] --> phase1[Phase1ReasoningEngine]
  phase1 --> phase2[Phase2DashboardIntegration]
  phase2 --> phase3[Phase3UXRefinement]
  phase0 --> scoringPipelines[ScoringPipelines]
  scoringPipelines --> insightCards[InsightCards]
  phase1 --> insightCards
```

Validation Plan

- Schema checks: FK integrity, unique constraints, nullability guards.
- RLS checks: owner/admin/member isolation and org boundary enforcement.
- Pipeline checks: sample signal -> diagnosis -> advisory -> structural remedy chain.
- Performance checks: index coverage on high-frequency filters (org, status, created_at, score windows).

Output Format To Provide

- A production-ready template in practical copy/paste form:
    - SQL Block A: foundational schema + security + baseline seeds
    - SQL Block B: realistic PMO sample data + score pipeline samples
    - concise “customize next” section for your exact business naming/weights and edge-function wiring

Tab 6

SYSTEM:
You are a senior AI systems architect and backend engineer.

Your single job is to convert raw product inputs into a fully specified AI operating system brain layer that sits between UI and backend.

Hard requirements:
- Be explicit and deterministic.
- Do not use vague language.
- Do not output summaries.
- Output executable system definitions, not advice.
- Think in execution steps only.
- Use language understandable by non-technical operators.
- Keep machine structures exact (JSON, formulas, pseudocode).

You MUST output EXACTLY these sections in this order:
SECTION 1 — INPUT CLASSIFICATION SYSTEM
SECTION 2 — CONTEXT RETRIEVAL SYSTEM (RAG)
SECTION 3 — DECISION ENGINE
SECTION 4 — FRAMEWORK EXECUTION MODEL
SECTION 5 — WORKFLOW ENGINE
SECTION 6 — MEMORY + LEARNING SYSTEM
SECTION 7 — FULL SYSTEM FLOW

Output format constraints:
1. Use deterministic JSON schemas (Draft 2020-12 style).
2. For every score, provide formula + variable definitions + threshold table.
3. For every pipeline, provide ordered steps with step IDs.
4. For every engine, provide pseudocode that is executable in sequence.
5. For every state machine, provide states, transitions, guards, retry policy, fallback policy.
6. For storage, provide concrete table schemas with field types and keys.
7. For APIs, provide concrete request/response schemas and status enums.
8. If an input field is missing, apply defaults from DEFAULT_POLICY and continue.
9. Never use these phrases: “you could”, “it depends”, “consider”, “typically”, “high-level”.
10. Do not ask clarifying questions. Use defaults when missing.

Determinism rules:
- Fixed execution order: Section1 -> Section2 -> Section3 -> Section4 -> Section5 -> Section6 -> Section7.
- Use explicit constants from CONSTANTS block.
- When two items tie on score, sort by lexical ascending ID.
- All probabilities/confidence values must be clamped to [0,1].
- All priority/risk scores must be clamped to [0,100].
- All ranking outputs must include score breakdown by term.

Non-technical language rules:
- Human-facing labels must avoid engineering jargon.
- Keep machine keys technical if needed, but include operator_label beside technical labels.
- Every section must include:
  - machine_view (strict structures)
  - operator_view (plain-language labels and instructions)

Failure behavior:
- If any section cannot be built from inputs, output a section-level ERROR object with:
  - error_code
  - missing_inputs
  - default_applied
  - next_action
- Continue generating remaining sections.

Return only the 7 required sections and their required structures. No preface. No appendix.

USER:
Use the exact generation contract below.

Required output sections and rules:
SECTION 1 — INPUT CLASSIFICATION SYSTEM
- machine_view must include:
  1) JSON Schema: InputEnvelopeSchema, ClassifiedInputSchema
  2) labels: full enum list
  3) scoring formulas:
     classification_score(label_i) =
       100 * (
         w_semantic * semantic_similarity_i +
         w_rule * rule_match_i +
         w_recency * recency_i +
         w_trust * trust_i
)
  4) thresholds table:
     - if score >= 85 => primary_label=label_i, confidence_tier="high"
     - if 65 <= score < 85 => confidence_tier="medium"
     - else confidence_tier="low"
  5) tie-breaker: lexical on label_id
- operator_view must include:
  - plain-language names for each label
  - one-line “what this means” text per label

SECTION 2 — CONTEXT RETRIEVAL SYSTEM (RAG)

- machine_view must include:
  1) chunking policy:
     - chunk_size_tokens=450
     - chunk_overlap_tokens=90
     - heading_aware=true
  2) embedding generation contract:
     - model_name
     - vector_dim
     - normalization method
  3) retrieval pipeline with step IDs:
     R1 ingest -> R2 chunk -> R3 embed -> R4 dense_search -> R5 sparse_search
     -> R6 merge -> R7 metadata_filter -> R8 rerank -> R9 final_context_pack
  4) ranking formula:
     retrieval_score(doc_j) =
       0.50*dense_sim_j +
       0.20*sparse_sim_j +
       0.15*recency_j +
       0.10*trust_j +
       0.05*policy_boost_j
  5) data structures for ChunkRecord, RetrievalCandidate, ContextPack
- operator_view must include:
  - plain “how context is chosen” steps with no jargon

SECTION 3 — DECISION ENGINE
- machine_view must include:
  1) deterministic if/then rule table
  2) framework execution order list
  3) formulas:
     priority_score = 100*(a*impact + b*urgency + c*business_value)
     risk_score = 100*(d*likelihood + e*severity)
     confidence_score = clamp(
       evidence_coverage - missing_fields*n_penalty - conflict_penalty, 0, 1
)
  4) execution pseudocode with fixed order and clamps
- operator_view must include:
  - plain labels: “Work urgency”, “Chance of problem”, “How sure the system is”

SECTION 4 — FRAMEWORK EXECUTION MODEL
- machine_view must include one executable object per framework:
  framework_name:
    inputs: [{id,type,required,default}]
    transform_steps: [{step_id,expression,validation}]
    output_schema: JSON Schema
    failure_conditions: [{condition,error_code,fallback}]

- operator_view must include:
  - simple step cards with inputs and expected output

SECTION 5 — WORKFLOW ENGINE
- machine_view must include:
  1) trigger definitions
  2) step sequence contract
  3) state machine:
     states=[queued,running,waiting_review,retrying,completed,failed,fallback]
     transitions with guards
  4) retry formula:
     retry_delay_seconds(attempt_n)=base_delay * (2^(attempt_n-1))
  5) fallback policy object
  6) JSON workflow templates
- operator_view must include:
  - status names and plain meaning

SECTION 6 — MEMORY + LEARNING SYSTEM
- machine_view must include:
  1) storage policy: what is persisted and retention windows
  2) schema: MemoryEvent, DecisionOutcome, FeedbackSignal, ModelAdjustment
  3) update formulas:
     new_weight = old_weight + lr * (observed_error * feature_value)
     trust_update = clamp(old_trust + alpha*positive - beta*negative, 0, 1)
  4) safety bounds and rollback conditions
- operator_view must include:
  - what the system remembers and when it updates behavior

SECTION 7 — FULL SYSTEM FLOW
- machine_view must include one complete trace:
  input_received -> classified -> context_packed -> decisions_scored
  -> framework_executed -> workflow_transitioned -> memory_written -> output_returned
- For each step include:
  - step_id
  - input_schema_ref
  - transformation_expression
  - output_schema_ref
  - persisted_records
  - failure_path
- operator_view must include:
  - plain-language timeline of the same flow

Supabase constraints:
{

```
  "supabase_contract_version": "v1",
  "extensions_required": ["pgcrypto", "vector", "pg_trgm"],
  "tables": [
    {
      "name": "kb_documents",
      "primary_key": "document_id",
      "columns": [
        {"name": "document_id", "type": "uuid", "default": "gen_random_uuid()"},
        {"name": "project_id", "type": "text", "not_null": true},
        {"name": "title", "type": "text", "not_null": true},
        {"name": "domain", "type": "text", "not_null": true},
        {"name": "source_type", "type": "text", "not_null": true},
        {"name": "source_uri", "type": "text", "not_null": false},
        {"name": "trust_level", "type": "numeric(4,3)", "not_null": true},
        {"name": "effective_from", "type": "date", "not_null": true},
        {"name": "effective_to", "type": "date", "not_null": true, "default": "'9999-12-31'::date"},
        {"name": "tags", "type": "jsonb", "not_null": true, "default": "'[]'::jsonb"},
        {"name": "created_at", "type": "timestamptz", "default": "now()"}
      ],
      "indexes": [
        {"name": "kb_documents_project_idx", "expr": "(project_id)"},
        {"name": "kb_documents_domain_idx", "expr": "(domain)"}
      ]
    },
    {
      "name": "kb_chunks",
      "primary_key": "chunk_id",
      "columns": [
        {"name": "chunk_id", "type": "uuid", "default": "gen_random_uuid()"},
        {"name": "document_id", "type": "uuid", "not_null": true, "references":
"kb_documents(document_id)"},
        {"name": "project_id", "type": "text", "not_null": true},
        {"name": "chunk_index", "type": "int", "not_null": true},
        {"name": "chunk_text", "type": "text", "not_null": true},
        {"name": "token_count", "type": "int", "not_null": true},
        {"name": "metadata", "type": "jsonb", "not_null": true, "default": "'{}'::jsonb"},
        {"name": "embedding", "type": "vector(1536)", "not_null": true},
        {"name": "created_at", "type": "timestamptz", "default": "now()"}
      ],
      "indexes": [
        {"name": "kb_chunks_project_idx", "expr": "(project_id)"},
        {"name": "kb_chunks_doc_chunk_uidx", "expr": "(document_id, chunk_index)", "unique":
true},
```

{"name": "kb_chunks_embedding_idx", "expr": "embedding vector_cosine_ops", "method": "ivfflat"}
]
},
{
"name": "brain_decisions",
"primary_key": "decision_id",
"columns": [
{"name": "decision_id", "type": "uuid", "default": "gen_random_uuid()"},
{"name": "request_id", "type": "text", "not_null": true},
{"name": "project_id", "type": "text", "not_null": true},
{"name": "classification_label", "type": "text", "not_null": true},
{"name": "priority_score", "type": "numeric(5,2)", "not_null": true},
{"name": "risk_score", "type": "numeric(5,2)", "not_null": true},
{"name": "confidence_score", "type": "numeric(5,4)", "not_null": true},
{"name": "decision_payload", "type": "jsonb", "not_null": true},
{"name": "created_at", "type": "timestamptz", "default": "now()"}
],
"indexes": [
{"name": "brain_decisions_request_idx", "expr": "(request_id)"},
{"name": "brain_decisions_project_created_idx", "expr": "(project_id, created_at desc)"}
]
},
{
"name": "workflow_runs",
"primary_key": "run_id",
"columns": [
{"name": "run_id", "type": "uuid", "default": "gen_random_uuid()"},
{"name": "request_id", "type": "text", "not_null": true},
{"name": "workflow_id", "type": "text", "not_null": true},
{"name": "state", "type": "text", "not_null": true},
{"name": "attempt_count", "type": "int", "not_null": true, "default": 0},
{"name": "last_error_code", "type": "text", "not_null": false},
{"name": "timeline", "type": "jsonb", "not_null": true, "default": "'[]'::jsonb"},
{"name": "updated_at", "type": "timestamptz", "default": "now()"}
],
"indexes": [
{"name": "workflow_runs_request_idx", "expr": "(request_id)"},
{"name": "workflow_runs_state_idx", "expr": "(state)"}
]
},
{
"name": "memory_events",
"primary_key": "memory_event_id",

```
      "columns": [
        {"name": "memory_event_id", "type": "uuid", "default": "gen_random_uuid()"},
        {"name": "request_id", "type": "text", "not_null": true},
        {"name": "event_type", "type": "text", "not_null": true},
        {"name": "event_payload", "type": "jsonb", "not_null": true},
        {"name": "quality_signal", "type": "numeric(5,4)", "not_null": false},
        {"name": "created_at", "type": "timestamptz", "default": "now()"}
      ],
      "indexes": [
        {"name": "memory_events_request_idx", "expr": "(request_id)"},
        {"name": "memory_events_type_idx", "expr": "(event_type)"}
      ]
    }
  ],
  "rpc_contracts": [
    {
      "name": "match_kb_chunks",
      "inputs": {
        "in_project_id": "text",
        "in_query_embedding": "vector(1536)",
        "in_top_k": "int",
        "in_domain_filter": "text[]"
      },
      "returns": {
        "chunk_id": "uuid",
        "document_id": "uuid",
        "chunk_text": "text",
        "cosine_similarity": "numeric",
        "trust_level": "numeric",
        "metadata": "jsonb"
      }
    }
  ],
  "rls_baseline": {
    "enabled": true,
    "policy_shape": "project_id = auth.jwt()->>'project_id'"
  }
}
```


Frontend + Cloudflare constraints:


```
{
  "api_version": "v1",
  "endpoints": [
    {
```

```
      "method": "POST",
      "path": "/brain/execute",
      "request_schema": {
        "type": "object",
        "required": ["request_id", "project_id", "input_payload"],
        "properties": {
          "request_id": {"type": "string"},
          "project_id": {"type": "string"},
          "input_payload": {"type": "object"},
          "operator_context": {"type": "object"},
          "workflow_id": {"type": "string"}
        }
      },
      "response_schema": {
        "type": "object",
        "required": ["request_id", "run_id", "state", "eta_seconds"],
        "properties": {
          "request_id": {"type": "string"},
          "run_id": {"type": "string"},
          "state": {
            "type": "string",
            "enum": ["queued", "running", "waiting_review", "retrying", "completed", "failed",
"fallback"]
          },
          "eta_seconds": {"type": "integer", "minimum": 0}
        }
      }
    },
    {
      "method": "GET",
      "path": "/brain/status/{run_id}",
      "request_schema": {
        "type": "object",
        "required": ["run_id", "project_id"],
        "properties": {
          "run_id": {"type": "string"},
          "project_id": {"type": "string"}
        }
      },
      "response_schema": {
        "type": "object",
        "required": ["run_id", "state", "attempt_count", "updated_at"],
        "properties": {
          "run_id": {"type": "string"},
```

```
        "state": {"type": "string"},
        "attempt_count": {"type": "integer"},
        "updated_at": {"type": "string", "format": "date-time"},
        "operator_status_label": {"type": "string"}
      }
    }
  },
  {
    "method": "GET",
    "path": "/brain/result/{run_id}",
    "request_schema": {
      "type": "object",
      "required": ["run_id", "project_id"],
      "properties": {
        "run_id": {"type": "string"},
        "project_id": {"type": "string"}
      }
    },
    "response_schema": {
      "type": "object",
      "required": ["run_id", "machine_view", "operator_view"],
      "properties": {
        "run_id": {"type": "string"},
        "machine_view": {"type": "object"},
        "operator_view": {"type": "object"},
        "trace_id": {"type": "string"}
      }
    }
  }
],
"cloudflare_worker_rules": {
  "idempotency_key_header": "x-idempotency-key",
  "request_timeout_seconds": 30,
  "max_payload_kb": 256,
  "retry_on_status": [408, 429, 500, 502, 503, 504],
  "circuit_breaker": {
    "failure_threshold": 5,
    "reset_window_seconds": 60
  }
},
"operator_language_rules": {
  "status_labels": {
    "queued": "Waiting to start",
    "running": "In progress",
```

```
      "waiting_review": "Needs human check",
      "retrying": "Trying again",
      "completed": "Done",
      "failed": "Stopped with issue",
      "fallback": "Used backup path"
    },
    "error_message_style": "one sentence, plain words, action-first"
  }
}
```

Validation checklist you must satisfy before you answer:
[STRUCTURE]
1. Exactly 7 sections exist, with exact titles and exact order.
2. Each section includes both machine_view and operator_view.
3. No extra intro, outro, summary, or appendix.

[DETERMINISM]
4. Every score has formula + variables + clamp/range.
5. Every threshold decision has explicit numeric bounds.
6. Every sequence has ordered step IDs.
7. Every tie-break has deterministic rule.

[SCHEMAS]
8. JSON schemas are syntactically valid and include required fields.
9. Enums are explicit (no open-ended text where states are needed).
10. Missing-field defaults are specified and applied.

[RAG]
11. Chunk policy includes size + overlap + metadata behavior.
12. Retrieval includes dense + sparse + rerank + final_k.
13. Ranking formula includes weighted terms totaling 1.0.

[DECISIONS]
14. Priority, risk, confidence formulas are present and bounded.
15. Rule table has explicit conditions and outputs.
16. Framework execution order is fixed and listed.

[WORKFLOW]
17. State machine includes states, transitions, guards.
18. Retry formula is explicit and parameterized.
19. Fallback behavior is explicit and trigger-based.

[MEMORY]
20. Stored entities and schemas are explicit.

21. Learning/update formulas are explicit with bounds.
22. Rollback or safety constraints are explicit.

[END-TO-END]
23. One complete trace from input to output is present.
24. Each trace step includes transform + intermediate state + persistence behavior.

[LANGUAGE]
25. Human-facing text avoids jargon.
26. No banned phrases appear.
27. Operator instructions are action-based and short.

Input packet:

```
{
  "meta": {
    "project_id": "string",
    "brain_layer_version": "v1.0.0",
    "generated_at_iso": "2026-03-26T00:00:00Z",
    "timezone": "UTC",
    "operator_language": "en",
    "constants_profile": "default_v1"
  },
  "knowledge_bases": [
    {
      "kb_id": "kb_001",
      "title": "Policy Rules",
      "domain": "operations",
      "source_type": "document",
      "source_uri": "string",
      "trust_level": 0.9,
      "effective_from": "2026-01-01",
      "effective_to": null,
      "text_content": "string",
      "tags": ["policy", "compliance"]
    }
  ],
  "frameworks": [
    {
      "framework_id": "fw_001",
      "framework_name": "Issue Priority Matrix",
      "purpose": "Decide work order",
      "input_fields": [
        {
          "field_id": "impact",
```

```
        "field_type": "number",
        "range": [0, 10],
        "required": true
      }
    ],
    "logic_steps": [
      {
        "step_id": "S1",
        "operation": "normalize",
        "expression": "(impact/10)"
      }
    ],
    "output_fields": [
      {
        "field_id": "priority_score",
        "field_type": "number",
        "range": [0, 100]
      }
    ],
    "constraints": [
      "priority_score must be integer"
    ]
  }
],
"features": [
  {
    "feature_id": "feat_001",
    "name": "Task Intake",
    "description": "Accept new requests from front end",
    "inputs": ["title", "description", "requested_by"],
    "outputs": ["request_id", "classification", "status"],
    "sla_minutes": 15,
    "risk_tolerance": "medium"
  }
],
"workflows": [
  {
    "workflow_id": "wf_001",
    "name": "New Request Handling",
    "trigger": {
      "type": "event",
      "event_name": "request_created"
    },
    "steps": [
```

```
      {
        "step_id": "W1",
        "name": "Classify Input",
        "max_retries": 2,
        "timeout_seconds": 30
      }
    ],
    "success_state": "completed",
    "failure_state": "failed"
  }
],
"integration": {
  "supabase": {
    "project_ref": "string",
    "region": "string",
    "pgvector_dimensions": 1536,
    "rls_mode": "enabled"
  },
  "frontend": {
    "surface": "cto.new",
    "edge_runtime": "cloudflare",
    "status_poll_interval_seconds": 3
  }
},
"DEFAULT_POLICY": {
  "missing_trust_level": 0.6,
  "missing_sla_minutes": 60,
  "missing_risk_tolerance": "medium",
  "missing_effective_to": "9999-12-31",
  "missing_tags": [],
  "missing_constraints": []
},
"CONSTANTS": {
  "classification_weights": {
    "semantic_match": 0.35,
    "rule_match": 0.30,
    "recency": 0.20,
    "trust_level": 0.15
  },
  "retrieval": {
    "top_k_dense": 25,
    "top_k_sparse": 25,
    "rerank_k": 20,
    "final_k": 8
```

```
    },
    "scores": {
      "priority_weight_impact": 0.5,
      "priority_weight_urgency": 0.3,
      "priority_weight_value": 0.2,
      "risk_weight_likelihood": 0.6,
      "risk_weight_severity": 0.4,
      "confidence_penalty_missing_fields": 0.08
    }
  }
}
```


Terminal instruction:
Wait for my input.

Tab 7

- Gemini (execution)
- Supabase (memory + prompts)
- Supabase Edge Functions (thinking layer)

---

1. Supabase Tables (Prompts + Memory)

✅ Create `prompts` table

```
create table prompts (
 id uuid primary key default gen_random_uuid(),
 name text unique,
 content text,
 version text default 'v1',
 created_at timestamp default now()
);
```

✅ Create `memory_logs` table

```
create table memory_logs (
 id uuid primary key default gen_random_uuid(),
 input text,
 classification jsonb,
 output jsonb,
 status text,
 created_at timestamp default now()
);
```

✅ Insert your FIRST prompts

```
insert into prompts (name, content) values
(
 'classifier',
 'You are MARTIN OS.

Classify the user input into:
- type: ticket | action | question | automation
- priority: low | medium | high
```

```
- intent: diagnose | create | fix | analyze

Return ONLY valid JSON.

Input:
{{input}}'
),
(
 'diagnostic',
 'You are a system diagnostics engine.

Analyze the issue and provide:
- root_cause
- recommended_fix
- steps

Return JSON only.

Context:
{{context}}

Input:
{{input}}'
);
```

---

2. Gemini Edge Function (CORE BRAIN)

This runs your thinking.

📁 File: `/supabase/functions/classifier/index.ts`

```
import { serve } from "https://deno.land/std/http/server.ts";

const GEMINI_API_KEY = Deno.env.get("GEMINI_API_KEY");

async function getPrompt(name: string) {
 const res = await
fetch(`${Deno.env.get("SUPABASE_URL")}/rest/v1/prompts?name=eq.${name}`, {
   headers: {
     apikey: Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!,
     Authorization: `Bearer ${Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!}`
```

```
  }
 });

 const data = await res.json();
 return data[0]?.content;
}

async function callGemini(prompt: string) {
 const res = await fetch(

`https://generativelanguage.googleapis.com/v1beta/models/gemini-pro:generateContent?key=${
GEMINI_API_KEY}`,
  {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      contents: [{ parts: [{ text: prompt }] }]
    })
  }
 );

 const data = await res.json();
 return data.candidates?.[0]?.content?.parts?.[0]?.text;
}

serve(async (req) => {
 const { input } = await req.json();

 // 1. Get prompt template
 let template = await getPrompt("classifier");

 // 2. Inject input
 const prompt = template.replace("{{input}}", input);

 // 3. Call Gemini
 const rawOutput = await callGemini(prompt);

 let parsed;
 try {
   parsed = JSON.parse(rawOutput);
 } catch {
   parsed = { error: "Invalid JSON", raw: rawOutput };
 }
```

```
 // 4. Store memory
 await fetch(`${Deno.env.get("SUPABASE_URL")}/rest/v1/memory_logs`, {
   method: "POST",
   headers: {
     "Content-Type": "application/json",
     apikey: Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!,
     Authorization: `Bearer ${Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!}`
   },
   body: JSON.stringify({
     input,
     classification: parsed,
     status: "classified"
   })
 });

 return new Response(JSON.stringify(parsed), {
   headers: { "Content-Type": "application/json" }
 });
});
```

---

3. Optimized Classifier Prompt (Gemini-Ready)

You already inserted one, but here's the **better version** (replace it):

```
You are MARTIN OS, an AI operations system.

Your task is to classify the input with strict structure.

Return ONLY JSON in this exact format:

{
 "type": "ticket | action | question | automation",
 "priority": "low | medium | high",
 "intent": "diagnose | create | fix | analyze",
 "confidence": 0-1
}

Rules:
- No explanations
- No extra text
```

- Always include all fields
- Be decisive (no nulls)

Input:
{{input}}

---

4. First Workflow (END-TO-END)

Now your system actually **does something useful**

Flow:

User Input
→ classifier function
→ IF type = ticket
→ run diagnostic function
→ return fix
→ store result

📁 Create SECOND Edge Function: `diagnose`

```
serve(async (req) => {
 const { input } = await req.json();

 // 1. Get diagnostic prompt
 let template = await getPrompt("diagnostic");

 // 2. Pull context (simple version)
 const context = "No prior context yet";

 const prompt = template
   .replace("{{input}}", input)
   .replace("{{context}}", context);

 // 3. Call Gemini
 const rawOutput = await callGemini(prompt);

 let parsed;
```

```
 try {
   parsed = JSON.parse(rawOutput);
 } catch {
   parsed = { error: "Invalid JSON", raw: rawOutput };
 }

 // 4. Store result
 await fetch(`${Deno.env.get("SUPABASE_URL")}/rest/v1/memory_logs`, {
   method: "POST",
   headers: {
     "Content-Type": "application/json",
     apikey: Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!,
     Authorization: `Bearer ${Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!}`
   },
   body: JSON.stringify({
     input,
     output: parsed,
     status: "diagnosed"
   })
 });

 return new Response(JSON.stringify(parsed), {
   headers: { "Content-Type": "application/json" }
 });
});
```

🔁 Simple Router Logic (you can add next)

Inside classifier response handling:

```
if (parsed.type === "ticket") {
 // call diagnose function
}
```

What You Now Have (This Is Big)

After this setup:

✅ Inputs get classified
✅ Prompts are dynamic (DB-driven)

✅ Gemini is wired into your system
✅ Results are stored (learning loop started)
✅ You have your FIRST real workflow

What To Do Immediately

1. Create both tables
2. Insert prompts
3. Deploy `classifier` function
4. Test with:

```
{
 "input": "My API is returning 500 errors"
}
```

What We're Building Now

You asked for all of it — so here it is:

✅ Router (auto-trigger workflows)

✅ Context Builder (memory retrieval)

✅ Self-improving system (learning loop)

✅ "Fix it" execution layer (not just suggestions)

---

1. ROUTER (AUTO-WORKFLOW ENGINE)

Right now your classifier just returns data.

👉 Now it will **trigger actions automatically**

Update your classifier function

Add this AFTER classification:

```
// ROUTER LOGIC

if (parsed.type === "ticket") {
 await fetch(`${Deno.env.get("SUPABASE_URL")}/functions/v1/diagnose`, {
   method: "POST",
   headers: { "Content-Type": "application/json" },
   body: JSON.stringify({ input })
 });
}

if (parsed.type === "action") {
 await fetch(`${Deno.env.get("SUPABASE_URL")}/functions/v1/execute`, {
   method: "POST",
   headers: { "Content-Type": "application/json" },
   body: JSON.stringify({ input })
 });
}
```

What this does

👉 Your system now **decides and acts automatically**

No UI logic needed.

---

2. CONTEXT BUILDER (REAL INTELLIGENCE)

Right now your system has no memory usage.

We fix that.

Add this function inside your edge functions:

```
async function getContext(input: string) {
 const res = await fetch(

`${Deno.env.get("SUPABASE_URL")}/rest/v1/memory_logs?select=*&limit=5&order=created_at.desc`,
   {
     headers: {
       apikey: Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!,
       Authorization: `Bearer ${Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!}`
     }
   }
 );

 const data = await res.json();

 return JSON.stringify(data);
}
```

Update your diagnostic function

Replace:

```
const context = "No prior context yet";
```

With:

```
const context = await getContext(input);
```

What this does

👉 Your system now:

- remembers past issues
- reuses past fixes
- gets smarter instantly

---

3. SELF-IMPROVING MEMORY LOOP

We upgrade your memory from logs → intelligence

Update `memory_logs` table

Add:

```
alter table memory_logs add column success boolean;
alter table memory_logs add column feedback text;
```

After execution, store outcomes:

```
await fetch(`${SUPABASE_URL}/rest/v1/memory_logs`, {
 method: "POST",
 headers: {...},
 body: JSON.stringify({
   input,
   output: parsed,
   status: "completed",
   success: true
 })
});
```

What this does

👉 Your system learns:

- what works
- what fails
- what to reuse

---

4. EXECUTION LAYER (THE “FIX IT” ENGINE)

Right now:
👉 your system suggests fixes

Now:
👉 it actually **does them**

Create `execute` edge function

```
serve(async (req) => {
 const { input } = await req.json();

 const prompt = `
You are an execution engine.

Convert the input into a structured action plan.

Return JSON:

{
 "actions": [
   {
     "type": "update_db | call_api | trigger_function",
     "target": "...",
     "data": {}
   }
 ]
}

Input:
${input}
`;

 const rawOutput = await callGemini(prompt);

 let parsed;
 try {
   parsed = JSON.parse(rawOutput);
 } catch {
   parsed = { error: "Invalid JSON", raw: rawOutput };
 }

 // Execute actions
 for (const action of parsed.actions || []) {
   if (action.type === "update_db") {
     await fetch(`${SUPABASE_URL}/rest/v1/${action.target}`, {
```

```
      method: "POST",
      headers: {...},
      body: JSON.stringify(action.data)
    });
  }

  if (action.type === "trigger_function") {
    await fetch(`${SUPABASE_URL}/functions/v1/${action.target}`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(action.data)
    });
  }
}

return new Response(JSON.stringify(parsed));
});
```

What this does

👉 Your system becomes:

- operator
- executor
- automation engine

---

5. OPTIONAL (BUT POWERFUL): SIMPLE “SIMILARITY SEARCH”

You don’t need vectors yet.

Use this:

```
async function findSimilar(input: string) {
 const res = await fetch(
   `${SUPABASE_URL}/rest/v1/memory_logs?input=ilike.%${input}%`,
   { headers: {...} }
 );
```

```
 return await res.json();
}
```

Add to context:

```
const similar = await findSimilar(input);
const context = JSON.stringify({ recent, similar });
```

🧠 What You Just Built (This Is Important)

You now have a system that:

THINKS

- classification
- reasoning

REMEMBERS

- logs
- outcomes
- past fixes

DECIDES

- routing
- workflow selection

ACTS

- executes fixes
- updates systems

🔥 Your System Is Now:

Not a chatbot.

👉 It's an **AI Operations Engine**

What You Should Do Next (Exact Order)

1. Add router logic

2. Add context function

3. Update diagnostic function

4. Create execute function

5. Test full flow

NEXT LEVEL SYSTEM (FULL UPGRADE)

1. Auto-Learning Prompt Optimizer

2. Confidence-Based Escalation

3. Multi-Model Routing (Gemini ↔ ChatGPT)

4. Decision Visibility Layer (so it's not a black box)

---

1. 🧠 AUTO-LEARNING PROMPT OPTIMIZER

Right now your prompts are static.

👉 We make them evolve automatically based on results.

Step 1 — Add prompt performance tracking

Update your `memory_logs` usage:

```
body: JSON.stringify({
 input,
 classification: parsed,
 output: result,
 prompt_used: "classifier_v1",
 success: true
})
```

Step 2 — Create `prompt_metrics` table

```
create table prompt_metrics (
 id uuid primary key default gen_random_uuid(),
 prompt_name text,
 success_rate float,
 total_runs int,
 last_updated timestamp default now()
);
```

Step 3 — Create optimizer function

This runs periodically (or manually at first):

```
serve(async () => {
 const res = await fetch(`${SUPABASE_URL}/rest/v1/memory_logs`, {
   headers: {...}
 });

 const logs = await res.json();

 const successCount = logs.filter(l => l.success).length;
 const rate = successCount / logs.length;

 await fetch(`${SUPABASE_URL}/rest/v1/prompt_metrics`, {
   method: "POST",
   headers: {...},
   body: JSON.stringify({
     prompt_name: "classifier",
```

```
    success_rate: rate,
    total_runs: logs.length
  })
});

 return new Response(JSON.stringify({ success: true }));
});
```

What this unlocks

👉 Later:

- auto-rewrite bad prompts
- A/B test prompts
- improve accuracy without touching code

---

2. ⚠️ CONFIDENCE-BASED ESCALATION

Right now your system blindly executes.

👉 That's dangerous.

Add this rule to your classifier output:

You already have:

"confidence": 0.82

Add control logic:

```
if (parsed.confidence < 0.6) {
 await fetch(`${SUPABASE_URL}/rest/v1/memory_logs`, {
   method: "POST",
   headers: {...},
   body: JSON.stringify({
     input,
```

```
    status: "needs_review",
    classification: parsed
  })
});

return new Response(JSON.stringify({
  message: "Escalated for review",
  classification: parsed
}));
}
```

What this does

👉 Prevents:

- bad automations
- incorrect fixes
- system damage

---

3. 🔀 MULTI-MODEL ROUTING (SMART MODEL USAGE)

Now we use BOTH:

- Gemini (fast/free)
- ChatGPT (higher reasoning when needed)

Create model router:

```
function chooseModel(input: string, classification: any) {
 if (classification.type === "ticket" && classification.priority === "high") {
   return "chatgpt"; // better reasoning
 }

 if (classification.intent === "analyze") {
   return "chatgpt";
 }
```

```
 return "gemini";
}
```

Update call layer:

```
const model = chooseModel(input, parsed);

let rawOutput;

if (model === "gemini") {
 rawOutput = await callGemini(prompt);
}

if (model === "chatgpt") {
 rawOutput = await callChatGPT(prompt);
}
```

What this does

👉 You:

- minimize cost
- maximize performance
- route intelligently

---

4. 👁️ DECISION VISIBILITY LAYER (CRITICAL)

Right now your system is a black box.

👉 That kills trust.

Add this to `memory_logs`:

alter table memory_logs add column decision jsonb;

Store full decision trace:

```
body: JSON.stringify({
 input,
 classification: parsed,
 decision: {
   chosen_model: model,
   triggered_workflow: parsed.type,
   confidence: parsed.confidence
 },
 output: result,
 status: "completed"
})
```

What this enables

Later:

- UI dashboard
- audit trail
- debugging
- explainability

🧠 YOUR SYSTEM NOW (FINAL FORM)

User Input
↓
Classifier (Gemini)
↓
Confidence Check
↓
Context Builder (Supabase)
↓
Model Router (Gemini / ChatGPT)
↓
LLM Execution
↓

Decision Engine
↓
Execution Layer (Fix / Act)
↓
Memory Logs (with full trace)
↓
Prompt Optimizer (improves system over time)

Tab 4

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

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

What You Should Do NEXT (Execution Plan)

TODAY:

- Add confidence check
- Add model router
- Log decisions

TOMORROW:

- Add optimizer tracking
- Start storing success/failure

AFTER:

- Build simple UI dashboard (even basic)

FINAL EVOLUTION ARCHITECTURE

You now add 4 major capabilities:

1. Self-Healing System

2. Autonomous Workflow Builder

3. Plugin / Extension System

4. Goal-Oriented Agent Loop (true autonomy)

---

1. 🔧 SELF-HEALING SYSTEM

Right now:

- system reacts to user input

👉 Now:

- system detects problems **without being asked**

Step 1 — Add “system_watch” function

This runs on a schedule (cron in Supabase or Cloudflare Workers):

```
serve(async () => {
 const res = await fetch(`${SUPABASE_URL}/rest/v1/memory_logs?status=eq.error`, {
   headers: {...}
 });

 const issues = await res.json();

 for (const issue of issues) {
   await fetch(`${SUPABASE_URL}/functions/v1/diagnose`, {
     method: "POST",
     body: JSON.stringify({ input: issue.input })
   });
 }

 return new Response(JSON.stringify({ scanned: issues.length }));
```

```
});
```

Step 2 — Detect patterns (repeat failures)

```
if (issues.length > 3) {

 // escalate OR auto-create permanent fix

}
```

What this does

👉 Your system:

- monitors itself
- detects repeated failures
- triggers fixes automatically

---

2. 🤖 AUTONOMOUS WORKFLOW BUILDER

Now the system creates **new workflows by itself**

Create `workflows` table

```
create table workflows (

 id uuid primary key default gen_random_uuid(),

 name text,

 trigger text,
```

```
 steps jsonb,
 created_at timestamp default now()
);
```

Workflow generation prompt

Store this in `prompts`:

```
You are an automation architect.

Based on repeated issues, create a reusable workflow.

Return JSON:

{
 "name": "...",
 "trigger": "...",
 "steps": [
   {"action": "...", "target": "..."}
 ]
}

Issues:
{{context}}
```

Function: `generate_workflow`

```
const context = await getContext("repeated failures");


const prompt = template.replace("{{context}}", context);


const workflow = JSON.parse(await callGemini(prompt));


await fetch(`${SUPABASE_URL}/rest/v1/workflows`, {
 method: "POST",
 headers: {...},
 body: JSON.stringify(workflow)
});
```

What this does

👉 Your system:

- learns patterns
- turns them into automations
- stores reusable workflows

---

3. 🔌 PLUGIN / EXTENSION SYSTEM

Now you make your system **expandable**

Create `tools` table

```
create table tools (
 id uuid primary key default gen_random_uuid(),
 name text,
 type text,
 endpoint text,
 created_at timestamp default now()
);
```

Example tools

name	type	endpoint
create_ticket	function	/functions/v1/ticket
send_email	api	https://api.email.com

Update execution engine

```
async function executeTool(action) {
 const res = await fetch(`${SUPABASE_URL}/rest/v1/tools?name=eq.${action.target}`, {
   headers: {...}
 });

 const tool = (await res.json())[0];
```

```
if (tool.type === "function") {
  await fetch(`${SUPABASE_URL}${tool.endpoint}`, {
    method: "POST",
    body: JSON.stringify(action.data)
  });
}

if (tool.type === "api") {
  await fetch(tool.endpoint, {
    method: "POST",
    body: JSON.stringify(action.data)
  });
}
}
```

What this does

👉 Your system becomes:

- modular
- extendable
- product-ready

---

4. 🎯 GOAL-ORIENTED AGENT LOOP (REAL AUTONOMY)

This is the biggest jump.

Instead of reacting…

👉 your system **pursues goals**

Create `goals` table

```
create table goals (
 id uuid primary key default gen_random_uuid(),
 goal text,
 status text default 'active',
 created_at timestamp default now()
);
```

Example goal

```
{
 "goal": "Reduce API errors by 50%"
}
```

Agent loop function

```
serve(async () => {
 const res = await fetch(`${SUPABASE_URL}/rest/v1/goals?status=eq.active`, {
   headers: {...}
 });
```

```
 const goals = await res.json();


 for (const goal of goals) {

   const context = await getContext(goal.goal);


   const prompt = `

You are an autonomous agent.


Goal:

${goal.goal}


Context:

${context}


Decide next action.


Return JSON:

{

 "action": "...",

 "reason": "...",

 "steps": []

}

`;
```

```
  const decision = JSON.parse(await callGemini(prompt));


  // Execute decision

  await executeAction(decision);

 }


 return new Response(JSON.stringify({ running: true }));

});
```

What this does

👉 Your system:

- sets direction
- evaluates progress
- takes action continuously

🧠 FINAL SYSTEM CAPABILITIES

After this, your system can:

✅ Detect problems automatically

✅ Fix issues without user input

✅ Create new workflows from patterns

✅ Expand via plugins/tools

✅ Pursue goals independently

⚠️ Reality Check (Important)

This is powerful — but:

👉 Without constraints, it can:

- loop endlessly
- take wrong actions
- degrade quality

Add guardrails (DO THIS)

1. Execution limits

```
if (actions.length > 5) stop;
```

2. Allowed tools only

```
if (!approvedTools.includes(action.target)) reject;
```

3. Confidence threshold

```
if (confidence < 0.7) escalate;
```

🧭 What You Do Next (Exact Plan)

Phase 1 (Now)

- Add workflows table
- Add tools table
- Add goal loop

Phase 2

- Turn on self-healing cron
- Start logging repeated failures

Phase 3

- Enable workflow generation

🔥 What You've Actually Built

Not exaggerating:

👉 This is the foundation of:

- AI ops platforms
- autonomous SaaS tools
- internal company copilots

GPT

You are a senior backend engineer.

Convert the following system design into a fully buildable implementation for a non-coder using:

- Supabase (database + vector store)
- API layer (Node.js or Python style functions)
- JSON-based workflow engine

OUTPUT MUST INCLUDE:

---

1. SUPABASE DATABASE SCHEMA

Provide:

- Tables
- Columns
- Data types
- Relationships

Include:

- classification_logs
- knowledge_base
- embeddings
- workflows
- decisions
- outcomes

---

2. VECTOR / RAG SETUP

Provide:

- Table structure for embeddings
- How to store vectors
- Example insert format
- Example query for similarity search

---

3. BACKEND FUNCTIONS (PSEUDO-CODE)

Provide functions for:

- classify_input()
- retrieve_context()
- run_framework()
- make_decision()
- execute_workflow()
- store_memory()

Each function must:

- include inputs
- include outputs
- include logic steps

---

4. WORKFLOW ENGINE (JSON)

Provide:

- Base workflow template
- Example workflows:
    - ticket handling
    - action execution
    - auto-fix

---

5. FRAMEWORK STORAGE FORMAT

Convert frameworks into:

- JSON
- Prompt templates (if needed)

---

6. API STRUCTURE

Define endpoints:

POST /input
POST /execute
GET /result

Include:

- request format
- response format

---

7. FULL EXECUTION EXAMPLE

Provide one full example:

User input → classification → retrieval → decision → execution → stored result

---

RULES:

- Everything must be copy-paste friendly
- No explanations unless necessary
- Prefer structured outputs
- Assume user does not code

Here is the system:
[PASTE gemini OUTPUT]

Tech + Miidle

FINAL PRODUCT ARCHITECTURE (ADMIN vs USER)

You are building:

> **MARTIN OS = One brain, multiple controlled access layers**

🔐 CORE IDEA

👉 **Same backend + same brain**
👉 Different **permissions, visibility, and power**

🧩 SYSTEM LAYERS

```
        MARTIN OS (CORE)
-------------------------------------
Brain (Middle Layer / Orchestrator)
Tech-Ops (Fix Engine)
PMO-Ops (Project Engine)
Automation Engine
-------------------------------------
   ↑                ↑
Admin Access      User Access
```

👑 1. ADMIN MODE (FULL CONTROL)

This is **your control center**

Admin gets:

🧠 Full Brain Access

- see classifications
- see routing decisions
- override decisions

🔧 Tech-Ops (FULL)

- view all system logs
- trigger fixes manually
- approve/reject auto-fixes
- see root causes

📊 PMO-Ops (FULL)

- manage all projects
- create/edit workflows
- assign tasks globally

⚙️ System Controls

- edit prompts (in Supabase)
- manage tools/plugins
- adjust thresholds (confidence, automation)
- enable/disable features

🧾 Full Visibility (CRITICAL)

- decision logs
- model used (Gemini vs ChatGPT)
- execution traces

Admin UI should show:

- “Why did the system do this?”

- “What will it do next?”
- “Override / Approve / Stop”

👤 2. USER MODE (PAID — ALL 3 APPS)

These users get **power — but controlled**

What they access:

🧠 Brain (LIMITED)

- they submit input
- they see results
- they DO NOT see full decision logic

🔧 Tech-Ops (LIMITED AUTO-FIX)

They can:

- submit issues
- receive fixes
- optionally enable auto-fix

BUT:

❌ cannot modify system-level tools
❌ cannot see internal logs
❌ cannot override core logic

📊 PMO-Ops (FULL FEATURE, LIMITED SCOPE)

They can:

- create projects

- manage tasks
- run workflows

BUT:

❌ only within their workspace
❌ no global system access

🤖 Automation Engine

They can:

- use workflows
- trigger automations
- use generated workflows

BUT:

❌ cannot create system-level automations (admin only)

🧱 MULTI-TENANT STRUCTURE (IMPORTANT)

You need **workspace isolation**

Add to ALL tables:

workspace_id uuid
user_id uuid
role text

Example:

`memory_logs`

workspace_id uuid,
user_id uuid,
visibility text default 'private'

What this does

👉 Keeps:

- user data separate
- admin global view intact

🔐 PERMISSION MODEL

Create a simple rule system:

Roles:

admin
power_user (paid full access)
user (basic/free)

Example logic in your edge functions:

```
if (role !== "admin" && action === "modify_prompts") {
 throw new Error("Unauthorized");
}
```

Routing Control:

```
if (role === "user") {
 disable_auto_execute = true;
}
```

🔌 NATIVE INTEGRATIONS (YOUR “3 APPS” MODEL)

You said:

> users who purchased all 3 apps

So structure it like this:

Apps (Modules)

1. Tech-Ops

2. PMO-Ops

3. Automation Engine

Add `subscriptions` table:

```
create table subscriptions (
 user_id uuid,
 tech_ops boolean default false,
 pmo_ops boolean default false,
 automation boolean default false
);
```

Gate access:

```
if (!user.tech_ops) {
 return "Upgrade required";
}
```

🧠 HOW THE BRAIN HANDLES USERS

Update your **brain function**:

```
if (classification.system === "tech_ops") {
 if (!user.tech_ops) {
   return new Response("Upgrade required");
 }

 return routeToTechOps();
}
```

This ensures:

👉 Same brain
👉 Different capabilities

👁️ VISIBILITY LAYERS (IMPORTANT FOR UX)

Admin sees:

```
{
 "input": "...",
 "classification": {...},
 "decision": {...},
 "model": "gemini",
 "execution": {...}
}
```

User sees:

```
{
 "result": "...",
 "status": "completed"
}
```

👉 Same system, different transparency

⚠️ CRITICAL DESIGN RULE

👉 NEVER duplicate systems for admin vs users

That leads to:

- bugs
- inconsistency
- impossible scaling

Instead:

👉 One system
👉 Controlled access

🧭 WHAT YOU SHOULD BUILD NEXT

1. Add `roles` + `subscriptions` tables

2. Update all edge functions with permission checks

3. Add workspace_id to all data

4. Update brain to enforce access

🔥 WHAT YOU NOW HAVE

After this:

👉 A **multi-tenant AI operating system**

- Admin = control layer
- Users = controlled execution
- Shared brain = consistent intelligence

If You Want the FINAL PIECE

Next step is what actually makes this sell:

💰 Monetization + UX layer

- pricing tiers
- onboarding flow
- “1-click deploy” like you wanted
- plugin marketplace feel

Tab 10

This updated plan acknowledges the "Tri-Native" architecture: **PMO-Ops**, **Tech-Ops**, and **miidle**. By using Edge Functions, you create a shared "Thinking Layer" (Apphia) that services all three integrations while keeping their data and execution logic distinct.

1. The Updated "Tri-Native" Build Plan

- **The Shared Intelligence (Edge):** One Edge Function acts as the "Central Processor." It doesn't care if a signal comes from a PMO project, a Tech-Ops support ticket, or a miidle "Build Story"—it uses the `source_app` tag to pull the correct framework.
- **The Execution Separation:** * **PMO-Ops:** Focuses on *Governance* (RACI, Risk Registers, Scopes).
    - **Tech-Ops:** Focuses on *Resolution* (Diagnostics, Root Cause, UDO paths).
    - **miidle:** Focuses on *Capture* (Work Graphs, Proof-of-Work, content generation).
- **The Workflow:** Database Webhooks on three different tables (or one unified signal table) trigger the same Edge Function. The function responds with a "brief plain English" explanation and then executes the specific native action required.

---

2. Updated Code & Scripts

Step A: The Unified Database Schema (SQL Editor)

We are adding a `source_app` column and a specific table for **miidle** to track "Work Events."

SQL
```
-- 1. Expanded Knowledge Base to support all 3 apps
CREATE TABLE IF NOT EXISTS public.knowledge_bases (
   id SERIAL PRIMARY KEY,
   source_app TEXT CHECK (source_app IN ('pmo_ops', 'tech_ops', 'miidle')),
   framework_name TEXT NOT NULL,
   description TEXT NOT NULL,
   decision_rules JSONB NOT NULL -- Logic gemini uses to decide "What to do"
);

-- 2. Unified Signal Table (The Trigger)
CREATE TABLE IF NOT EXISTS public.operational_signals (
   id SERIAL PRIMARY KEY,
   source_app TEXT NOT NULL, -- 'pmo_ops', 'tech_ops', or 'miidle'
   signal_type TEXT NOT NULL, -- e.g., 'capacity_gap', 'system_fault', 'work_capture'
   context_data JSONB NOT NULL, -- The raw data from the specific app
   status TEXT DEFAULT 'unprocessed',
   created_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);
```

```
-- 3. miidle Specific: Work Graph & Proof-of-Work
CREATE TABLE IF NOT EXISTS public.miidle_events (
   id SERIAL PRIMARY KEY,
   user_id UUID,
   event_type TEXT, -- 'code_push', 'meeting_notes', 'design_update'
   raw_input TEXT,
   structured_content JSONB, -- gemini will turn raw_input into this
   created_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);
```

Step B: The "Apphia" Edge Function (The Logic)

This function now handles the context for all three apps. Replace your `supabase/functions/apphia-engine/index.ts` with this:

TypeScript
```
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

const anthropic_key = Deno.env.get('ANTHROPIC_API_KEY');

serve(async (req) => {
  const payload = await req.json();
  const record = payload.record;

  const supabase = createClient(
    Deno.env.get('SUPABASE_URL') ?? '',
    Deno.env.get('SUPABASE_SERVICE_ROLE_KEY') ?? ''
  );

  // 1. Context Switcher: Determine which 'Thinking Engine' to use
  let systemInstructions = "";
  if (record.source_app === 'pmo_ops') {
    systemInstructions = "Focus on governance, risk scores, and project alignment.";
  } else if (record.source_app === 'tech_ops') {
    systemInstructions = "Focus on technical root cause analysis and rapid ticket resolution.";
  } else if (record.source_app === 'miidle') {
    systemInstructions = "Focus on 'Proof-of-Work'. Convert raw updates into a structured Build
Story.";
  }

  try {
    // 2. Knowledge Retrieval
```

```
    const { data: frameworks } = await supabase
      .from('knowledge_bases')
      .select('*')
      .eq('source_app', record.source_app);

    // 3. gemini Call
    const response = await fetch('https://api.anthropic.com/v1/messages', {
      method: 'POST',
      headers: {
        'x-api-key': anthropic_key || '',
        'anthropic-version': '2023-06-01',
        'content-type': 'application/json',
      },
      body: JSON.stringify({
        model: "gemini-3-5-sonnet-20241022",
        max_tokens: 1000,
        system: `You are MARTIN OS. ${systemInstructions} Use these frameworks:
${JSON.stringify(frameworks)}.
                Provide a brief plain English summary and a JSON execution object.`,
        messages: [{ role: "user", content: `Context: ${JSON.stringify(record.context_data)}` }],
      }),
    });

    const result = await response.json();
    const thought = result.content[0].text;

    // 4. Multi-App Execution
    if (record.source_app === 'miidle') {
      // Create a "Build Story" entry automatically
      await supabase.from('miidle_events').insert({
        raw_input: JSON.stringify(record.context_data),
        structured_content: { summary: thought }
      });
    }

    // Save the reasoning back to the signal
    await supabase.from('operational_signals').update({
      status: 'processed',
      reasoning_output: thought
    }).eq('id', record.id);

    return new Response(JSON.stringify({ status: "success" }));
  } catch (err) {
    return new Response(JSON.stringify({ error: err.message }), { status: 500 });
```

```
  }
});
```

3. Implementation Step for Cursor

1. **Framework Injection:** You need to tell gemini how to handle **miidle**. In your `knowledge_bases` table, add a row for `source_app: 'miidle'` with a framework called "Build Story Engine."
2. **Deployment:** Run `supabase functions deploy apphia-engine` in your Cursor terminal.
3. **Webhook:** In Supabase, point your `operational_signals` webhook to this function.

This setup ensures that when you're working in **PMO-Ops**, Apphia acts like a COO; in **Tech-Ops**, she acts like a Lead Engineer; and in **miidle**, she acts like a Content/Documentation lead—all using the same free/low-cost Edge Function infrastructure.

_

Gemini said

To power the "brain" of your tri-native ecosystem, you need a library of objective formulas. These functions allow the **Apphia Engine** to calculate hard numbers before **gemini** adds the qualitative reasoning.

Here are the essential formulas for **PMO-Ops**, **Tech-Ops**, and **miidle**, structured to be plugged directly into your Supabase Edge Functions or stored as `decision_rules` in your database.

---

1. PMO-Ops: The Governance & Strategy Brain

These formulas focus on resource allocation, risk mitigation, and "Strategic ROI."

- **Executive Priority Score (EPS):**
  - *Formula:* `(Impact * 0.4) + (Urgency * 0.3) + (Strategic Alignment * 0.3)`
  - *Purpose:* Determines which initiative gets budget and attention first.
- **Risk Magnitude Score (RMS):**
  - *Formula:* `Probability (1-5) * Impact (1-5)`
  - *Logic:* Any score >15 triggers an automatic "Mitigation Plan" requirement in the UI.
- **Capacity Strain Index (CSI):**

    - *Formula:* `(Allocated Hours / Available Hours) * (1 + Context Switching Penalty)`
    - *Logic:* If CSI >0.85, Apphia flags a "Burnout Risk" signal.
- **Operational Maturity Score (OMS):**
    - *Formula:* `(Governance Compliance + Execution Discipline + Process Documentation) / 3`
    - *Purpose:* Scores how well a department is running its internal "machinery."

---

2. Tech-Ops: The Efficiency & Resolution Brain

These focus on the "UDO Path" (Understand, Diagnose, Optimize) and system health.

- **Resolution Efficiency Ratio (RER):**
    - *Formula:* `(Case Complexity Score / Resolution Time in Minutes)`
    - *Purpose:* Measures if senior engineers are wasting time on simple tasks or if junior staff are "stuck" on high-complexity issues.
- **Root Cause Confidence (RCC):**
    - *Formula:* `(Data Point Density * Pattern Match Strength) / Diagnostic Tiers Traversed`
    - *Logic:* Tells the user if the AI's diagnosis is a "Guess" ($<40\%$) or a "Certainty" ($>85\%$).
- **System Fragility Index (SFI):**
    - *Formula:* `(Error Rate * Latency) + (Days Since Last Documentation Update)`
    - *Purpose:* Identifies "Technical Debt" hotspots that are likely to break soon.
- **Automation Leverage Factor (ALF):**
    - *Formula:* `(Manual Steps Replaced / Total Process Steps) * Frequency of Task`
    - *Purpose:* Calculates the ROI of building a new Edge Function or workflow.

---

3. miidle: The Content & Proof-of-Work Brain

These power the "Work Graph" and the "Build Story Engine" by converting raw work into social/spectator value.

- **Proof-of-Work (PoW) Density:**
  - *Formula:* `(Actions Recorded + Commits/Files + Time Spent) / Milestone Duration`
  - *Purpose:* Validates that a "Build Story" is authentic and not just AI-generated fluff.
- **Spectator Engagement Velocity (SEV):**
  - *Formula:* `(Views / Time) * (Comment Sentiment + Share Rate)`
  - *Purpose:* Determines which build stories move to the "Featured" feed.
- **Content "Remix" Potential:**
  - *Formula:* `(Technical Depth * Narrative Clarity) / Jargon Density`
  - *Logic:* gemini uses this to decide if a piece of work should be turned into a video script, a blog post, or a technical white paper.
- **Virality Hook Score:**
  - *Formula:* `(Emotional "Messiness" + Successful Outcome) * Visual Frequency`
  - *Purpose:* Identifies the "Messy Middle" moments that perform best on social feeds (people love seeing how things broke before they were fixed).

How to Implement these in TypeScript (Edge Functions)

You can create a `lib/formulas.ts` file in your Supabase project to keep these consistent across all three apps:

TypeScript

```
export const ApphiaBrain = {
  // PMO-Ops
  calculatePriority: (i: number, u: number, a: number) => (i * 0.4) + (u * 0.3) + (a * 0.3),

  // Tech-Ops
  calculateResolutionEfficiency: (complexity: number, mins: number) => complexity / mins,

  // miidle
  calculatePoWDensity: (actions: number, time: number) => actions / Math.log(time + 1),
```

```
  // Global Multi-App Logic: The "Context Switcher"
  getRequiredFramework: (sourceApp: string, signalType: string)
=> {
    const map = {
      pmo_ops: signalType === 'risk' ? 'Risk Register' : 'RACI
Matrix',
      tech_ops: 'Root Cause Analysis (5 Whys)',
      miidle: 'Build Story Mapping'
    };
    return map[sourceApp] || 'Standard Operational Audit';
  }
};
```

The "Thinking" Prompt for gemini

When your Edge Function calls gemini, you should pass these calculated numbers first.

Example Prompting Strategy:

> "The signal from **Tech-Ops** has a **System Fragility Index of 8.2/10** and a **Resolution Efficiency of 0.2**. Using the **Technical Debt Framework**, provide a 1-sentence plain English summary for the Executive and a structural remedy."

This ensures gemini isn't "guessing" how bad the situation is—it is reacting to the hard math generated by your code.

Tab 11

build a **thinking and execution layer** for your app using AI agents. Let's break this down step by step. Since you already have **backend (Supabase), Cursor, CTO.new, and Cloudflare**, we can leverage these. I'll map out how to create an **agent-based brain** that handles reasoning (thinking) and action (execution).

---

1. Understanding the Layers

1. **Thinking Layer (Cognitive/Reasoning Layer)**
    - This is where AI interprets data, plans steps, and decides what actions to take.
    - It uses:
        - **Knowledge base**: your frameworks, KBs, documentation.
        - **Contextual memory**: past actions, state of users/projects.
        - **Reasoning agents**: multiple agents can specialize (e.g., one for project management, one for knowledge retrieval, one for user interaction).
2. **Execution Layer (Action Layer)**
    - This is where the AI **executes tasks**: updating databases, sending emails, triggering APIs, interacting with users.
    - It often uses:
        - **Workflow agents**: AI decides which function/API to call next.
        - **Middle-layer orchestrator**: manages agent outputs and ensures correct sequencing.

---

2. Using Agents

You can structure your thinking/execution layer using **AI agents**:

Agent Type	Role	Tools / Methods
Reasoning Agent	Interprets prompts, decides next steps	GPT-4/5, Claude, Gemini, LangChain
Retriever Agent	Pulls data from KBs, Supabase, or external sources	Vector DBs (Supabase, Pinecone)
Executor Agent	Performs actions (update DB, send emails, call APIs)	Supabase functions, Webhooks, CTO.new actions
Orchestrator Agent	Oversees workflows, ensures sequence	Cursor, CTO.new workflows, custom prompt chains

---

3. Workflow Example

1. **User asks a question / triggers a task** → input goes to **Reasoning Agent**.
2. **Reasoning Agent** queries **Retriever Agent** for relevant info.
3. **Reasoning Agent** decides which actions are needed → sends commands to **Executor Agent**.
4. **Executor Agent** performs tasks (update DB, trigger API, return results).
5. **Orchestrator Agent** ensures errors are handled, retries if needed.

---

4. How to Build This with Prompts (No-Code Approach)

You can use **prompt-driven agent orchestration** without coding:

1. **Define agent roles** in plain language:

   ```
   You are a Reasoning Agent. Your job is to:
   1. Understand user input
   2. Decide what information is needed
   3. Delegate tasks to Executor or Retriever Agents
   4. Return structured output
   ```

2. **Create structured output templates**:

   ```
   {
    "action": "update_db",
    "table": "tasks",
    "row_data": {
      "task_name": "Follow up with client",
      "due_date": "2026-03-27"
    },
    "response_to_user": "Task has been scheduled."
   }
   ```

3. **Feed these prompts to GPT/Gemini** and have them return **JSON commands**, which your backend can execute.
   - This approach avoids hardcoding logic—AI decides the sequence.

---

5. Tools to Implement Agents Without Code

Function	Tool	Notes
AI reasoning	GPT-4/5, Gemini	Can be prompted to act as reasoning agent
Agent orchestration	Cursor, CTO.new	Can manage workflow pipelines and chaining AI prompts
Database / storage	Supabase	Store knowledge, intermediate results, user state
Function execution	Supabase Functions, Cloudflare Workers	Execute tasks in backend safely
Knowledge retrieval	Supabase + vector embeddings	Allows agents to “look up” info efficiently

---

6. Example Prompt Chain

Step 1: Reasoning Agent

You are the AI brain. Analyze this task:
"Schedule a follow-up call with the client next week."

1. Check if client exists in DB.
2. If not, add client.
3. Schedule the call.
4. Return JSON with action instructions.

Step 2: Executor Agent

- Receives JSON → executes DB insert / API call → returns status.

Step 3: Orchestrator Agent

- Ensures sequence, logs errors, retries if needed.

Code

```sql
-- Brain layer foundation for ChatGPT + Gemini orchestration
-- Deterministic storage, retrieval, routing, and learning tables.

create extension if not exists vector;
create extension if not exists pg_trgm;
create extension if not exists pgcrypto;

create table if not exists public.brain_prompts (
 id uuid primary key default gen_random_uuid(),
 name text not null unique,
 description text,
 active_version text not null default 'v1',
 created_at timestamptz not null default now(),
 updated_at timestamptz not null default now()
);

create table if not exists public.brain_prompt_versions (
 id uuid primary key default gen_random_uuid(),
 prompt_id uuid not null references public.brain_prompts(id) on delete cascade,
 version text not null,
 content text not null,
 model_target text not null default 'gemini' check (model_target in ('gemini',
'chatgpt', 'both')),
 success_rate numeric(6,4),
 total_runs integer not null default 0,
 is_active boolean not null default false,
 created_at timestamptz not null default now(),
 unique (prompt_id, version)
);

create table if not exists public.brain_frameworks (
 id uuid primary key default gen_random_uuid(),
 profile_id uuid not null references public.profiles(id) on delete cascade,
 organization_id uuid references public.organizations(id) on delete cascade,
 framework_name text not null,
 framework_json jsonb not null default '{}',
 version text not null default 'v1',
 active boolean not null default true,
 created_at timestamptz not null default now(),
 updated_at timestamptz not null default now()
);
```

```sql
create table if not exists public.brain_workflows (
 id uuid primary key default gen_random_uuid(),
 profile_id uuid not null references public.profiles(id) on delete cascade,
 organization_id uuid references public.organizations(id) on delete cascade,
 workflow_key text not null,
 trigger_type text not null default 'event',
 workflow_json jsonb not null default '{}',
 active boolean not null default true,
 created_at timestamptz not null default now(),
 updated_at timestamptz not null default now(),
 unique (profile_id, workflow_key)
);

create table if not exists public.brain_runs (
 id uuid primary key default gen_random_uuid(),
 request_id text not null,
 profile_id uuid not null references public.profiles(id) on delete cascade,
 organization_id uuid references public.organizations(id) on delete cascade,
 input_payload jsonb not null,
 state text not null default 'queued' check (state in ('queued', 'running',
'waiting_review', 'retrying', 'completed', 'failed', 'fallback')),
 chosen_model text check (chosen_model in ('gemini', 'chatgpt')),
 confidence numeric(6,4),
 trace jsonb not null default '[]'::jsonb,
 created_at timestamptz not null default now(),
 updated_at timestamptz not null default now(),
 unique (profile_id, request_id)
);

create table if not exists public.brain_decision_runs (
 id uuid primary key default gen_random_uuid(),
 run_id uuid not null references public.brain_runs(id) on delete cascade,
 profile_id uuid not null references public.profiles(id) on delete cascade,
 classification jsonb not null default '{}',
 context_pack jsonb not null default '{}',
 decision jsonb not null default '{}',
 priority_score numeric(6,2),
 risk_score numeric(6,2),
 confidence_score numeric(6,4),
 created_at timestamptz not null default now()
);
```

```sql
create table if not exists public.brain_workflow_runs (
 id uuid primary key default gen_random_uuid(),
 run_id uuid not null references public.brain_runs(id) on delete cascade,
 profile_id uuid not null references public.profiles(id) on delete cascade,
 workflow_key text not null,
 state text not null check (state in ('queued', 'running', 'waiting_review',
'retrying', 'completed', 'failed', 'fallback')),
 attempt_count integer not null default 0,
 last_error_code text,
 timeline jsonb not null default '[]'::jsonb,
 created_at timestamptz not null default now(),
 updated_at timestamptz not null default now()
);

create table if not exists public.brain_outcomes (
 id uuid primary key default gen_random_uuid(),
 run_id uuid not null references public.brain_runs(id) on delete cascade,
 profile_id uuid not null references public.profiles(id) on delete cascade,
 success boolean not null default false,
 feedback text,
 time_to_resolve_seconds integer,
 output_payload jsonb not null default '{}',
 created_at timestamptz not null default now()
);

create table if not exists public.brain_memory_logs (
 id uuid primary key default gen_random_uuid(),
 run_id uuid references public.brain_runs(id) on delete set null,
 request_id text not null,
 profile_id uuid not null references public.profiles(id) on delete cascade,
 organization_id uuid references public.organizations(id) on delete cascade,
 stage text not null,
 status text not null,
 input_payload jsonb,
 output_payload jsonb,
 classification jsonb,
 decision_trace jsonb,
 chosen_model text check (chosen_model in ('gemini', 'chatgpt')),
 confidence numeric(6,4),
 success boolean,
 created_at timestamptz not null default now()
);
```

```sql
create table if not exists public.brain_knowledge_documents (
 id uuid primary key default gen_random_uuid(),
 profile_id uuid not null references public.profiles(id) on delete cascade,
 organization_id uuid references public.organizations(id) on delete cascade,
 title text not null,
 domain text not null default 'general',
 source_type text not null default 'document',
 source_uri text,
 trust_level numeric(4,3) not null default 0.600,
 metadata jsonb not null default '{}'::jsonb,
 created_at timestamptz not null default now()
);

create table if not exists public.brain_knowledge_chunks (
 id uuid primary key default gen_random_uuid(),
 document_id uuid not null references public.brain_knowledge_documents(id) on delete
cascade,
 profile_id uuid not null references public.profiles(id) on delete cascade,
 organization_id uuid references public.organizations(id) on delete cascade,
 chunk_index integer not null,
 chunk_text text not null,
 token_count integer not null default 0,
 metadata jsonb not null default '{}'::jsonb,
 embedding vector(1536),
 created_at timestamptz not null default now(),
 unique (document_id, chunk_index)
);

create table if not exists public.brain_tool_registry (
 id uuid primary key default gen_random_uuid(),
 profile_id uuid not null references public.profiles(id) on delete cascade,
 tool_name text not null,
 tool_type text not null check (tool_type in ('update_db', 'trigger_function',
'call_api')),
 target text not null,
 active boolean not null default true,
 created_at timestamptz not null default now(),
 unique (profile_id, tool_name)
);

create or replace function public.brain_set_updated_at()
```

```
returns trigger
language plpgsql
as $$
begin
 new.updated_at = now();
 return new;
end;
$$;

drop trigger if exists trg_brain_prompts_updated_at on public.brain_prompts;
create trigger trg_brain_prompts_updated_at before update on public.brain_prompts
for each row execute function public.brain_set_updated_at();

drop trigger if exists trg_brain_frameworks_updated_at on public.brain_frameworks;
create trigger trg_brain_frameworks_updated_at before update on
public.brain_frameworks
for each row execute function public.brain_set_updated_at();

drop trigger if exists trg_brain_workflows_updated_at on public.brain_workflows;
create trigger trg_brain_workflows_updated_at before update on public.brain_workflows
for each row execute function public.brain_set_updated_at();

drop trigger if exists trg_brain_runs_updated_at on public.brain_runs;
create trigger trg_brain_runs_updated_at before update on public.brain_runs
for each row execute function public.brain_set_updated_at();

drop trigger if exists trg_brain_workflow_runs_updated_at on
public.brain_workflow_runs;
create trigger trg_brain_workflow_runs_updated_at before update on
public.brain_workflow_runs
for each row execute function public.brain_set_updated_at();

create index if not exists idx_brain_prompt_versions_prompt_active on
public.brain_prompt_versions(prompt_id, is_active);
create index if not exists idx_brain_frameworks_profile on
public.brain_frameworks(profile_id, active);
create index if not exists idx_brain_workflows_profile on
public.brain_workflows(profile_id, active);
create index if not exists idx_brain_runs_profile_request on
public.brain_runs(profile_id, request_id);
create index if not exists idx_brain_runs_state on public.brain_runs(state, updated_at
desc);
```

```sql
create index if not exists idx_brain_decision_runs_run on
public.brain_decision_runs(run_id);
create index if not exists idx_brain_workflow_runs_run on
public.brain_workflow_runs(run_id);
create index if not exists idx_brain_memory_logs_profile_created on
public.brain_memory_logs(profile_id, created_at desc);
create index if not exists idx_brain_memory_logs_request on
public.brain_memory_logs(request_id);
create index if not exists idx_brain_chunks_embedding on public.brain_knowledge_chunks
using ivfflat (embedding vector_cosine_ops) with (lists = 100);
create index if not exists idx_brain_chunks_text on public.brain_knowledge_chunks
using gin (chunk_text gin_trgm_ops);

create or replace function public.match_brain_chunks(
 in_profile_id uuid,
 in_query_embedding vector(1536),
 in_top_k int default 8
)
returns table (
 chunk_id uuid,
 document_id uuid,
 chunk_text text,
 cosine_similarity numeric,
 metadata jsonb
)
language sql
stable
as $$
 select
   c.id as chunk_id,
   c.document_id,
   c.chunk_text,
   (1 - (c.embedding <=> in_query_embedding))::numeric as cosine_similarity,
   c.metadata
 from public.brain_knowledge_chunks c
 where c.profile_id = in_profile_id and c.embedding is not null
 order by c.embedding <=> in_query_embedding
 limit greatest(in_top_k, 1);
$$;

alter table public.brain_prompts enable row level security;
alter table public.brain_prompt_versions enable row level security;
```

```sql
alter table public.brain_frameworks enable row level security;
alter table public.brain_workflows enable row level security;
alter table public.brain_runs enable row level security;
alter table public.brain_decision_runs enable row level security;
alter table public.brain_workflow_runs enable row level security;
alter table public.brain_outcomes enable row level security;
alter table public.brain_memory_logs enable row level security;
alter table public.brain_knowledge_documents enable row level security;
alter table public.brain_knowledge_chunks enable row level security;
alter table public.brain_tool_registry enable row level security;

drop policy if exists "brain_prompts_own" on public.brain_prompts;
create policy "brain_prompts_own" on public.brain_prompts
for all
using (auth.uid() is not null)
with check (auth.uid() is not null);

drop policy if exists "brain_prompt_versions_own" on public.brain_prompt_versions;
create policy "brain_prompt_versions_own" on public.brain_prompt_versions
for all
using (auth.uid() is not null)
with check (auth.uid() is not null);

drop policy if exists "brain_frameworks_own" on public.brain_frameworks;
create policy "brain_frameworks_own" on public.brain_frameworks
for all using (profile_id = auth.uid()) with check (profile_id = auth.uid());

drop policy if exists "brain_workflows_own" on public.brain_workflows;
create policy "brain_workflows_own" on public.brain_workflows
for all using (profile_id = auth.uid()) with check (profile_id = auth.uid());

drop policy if exists "brain_runs_own" on public.brain_runs;
create policy "brain_runs_own" on public.brain_runs
for all using (profile_id = auth.uid()) with check (profile_id = auth.uid());

drop policy if exists "brain_decision_runs_own" on public.brain_decision_runs;
create policy "brain_decision_runs_own" on public.brain_decision_runs
for all using (profile_id = auth.uid()) with check (profile_id = auth.uid());

drop policy if exists "brain_workflow_runs_own" on public.brain_workflow_runs;
create policy "brain_workflow_runs_own" on public.brain_workflow_runs
for all using (profile_id = auth.uid()) with check (profile_id = auth.uid());
```

```
drop policy if exists "brain_outcomes_own" on public.brain_outcomes;
create policy "brain_outcomes_own" on public.brain_outcomes
for all using (profile_id = auth.uid()) with check (profile_id = auth.uid());

drop policy if exists "brain_memory_logs_own" on public.brain_memory_logs;
create policy "brain_memory_logs_own" on public.brain_memory_logs
for all using (profile_id = auth.uid()) with check (profile_id = auth.uid());

drop policy if exists "brain_kb_docs_own" on public.brain_knowledge_documents;
create policy "brain_kb_docs_own" on public.brain_knowledge_documents
for all using (profile_id = auth.uid()) with check (profile_id = auth.uid());

drop policy if exists "brain_kb_chunks_own" on public.brain_knowledge_chunks;
create policy "brain_kb_chunks_own" on public.brain_knowledge_chunks
for all using (profile_id = auth.uid()) with check (profile_id = auth.uid());

drop policy if exists "brain_tools_own" on public.brain_tool_registry;
create policy "brain_tools_own" on public.brain_tool_registry
for all using (profile_id = auth.uid()) with check (profile_id = auth.uid());

insert into public.brain_prompts (name, description, active_version)
values
 ('classifier', 'Classifies input into deterministic routing labels', 'v1'),
 ('context_builder', 'Builds context pack from memory and knowledge', 'v1'),
 ('decision_engine', 'Produces action plan and confidence with strict JSON', 'v1'),
 ('executor', 'Converts decision output to executable action list', 'v1')
on conflict (name) do nothing;

insert into public.brain_prompt_versions (prompt_id, version, content, model_target,
is_active)
select p.id, 'v1',
'Return JSON only:
{"type":"ticket|action|question|automation","priority":"low|medium|high","intent":"dia
gnose|create|fix|analyze","domain":"tech|pmo|ops|general","complexity":"low|medium|hig
h","confidence":0.0}',
'both', true
from public.brain_prompts p
where p.name = 'classifier'
and not exists (
 select 1 from public.brain_prompt_versions pv
 where pv.prompt_id = p.id and pv.version = 'v1'
```

```
);
```